

Puma-Benutzerhandbuch

Sprache: Deutsch

Zielgruppe: Administratoren und Betreiber

1. Zweck und Geltungsbereich

Puma ist das zentrale System für Authentifizierung und Berechtigungsverwaltung mehrerer Anwendungen. Benutzer melden sich gegenüber Puma an; Anwendungen entscheiden anhand der von Puma gelieferten Berechtigungen, welche Funktionen freigeschaltet werden. Dadurch müssen Benutzer, Rollen und Gruppen nicht in jeder Anwendung separat gepflegt werden.

Dieses Handbuch beschreibt:

- den Puma-Server mit SQLite- oder PostgreSQL-Datenbank,
- lokale Benutzer, Rollen, Gruppen und produktbezogene Berechtigungen,
- die Einbindung eigener Qt/C++-Anwendungen über `AuthClientSdk`,
- die Einbindung eigener Server über `AuthServerSdk`,
- Windows-Domänenanmeldung über die in `ImtCore` implementierte LDAP-Schicht,
- Personal Access Tokens (PATs) für nicht interaktive Zugriffe,
- Passwortrichtlinie, Kontostatus und Sperre nach Fehlversuchen,
- typische Betriebs-, Sicherheits- und Fehlerfälle.

Die Beschreibung basiert auf Puma und der zugrunde liegenden Authentifizierungsimplementierung in `ImagingTools/ImtCore`. Sie beschreibt den Stand der im Quellcode vorhandenen Funktionen; konkrete Menünamen können je nach eingebetteter Administrationsoberfläche abweichen.

2. Puma auf einen Blick

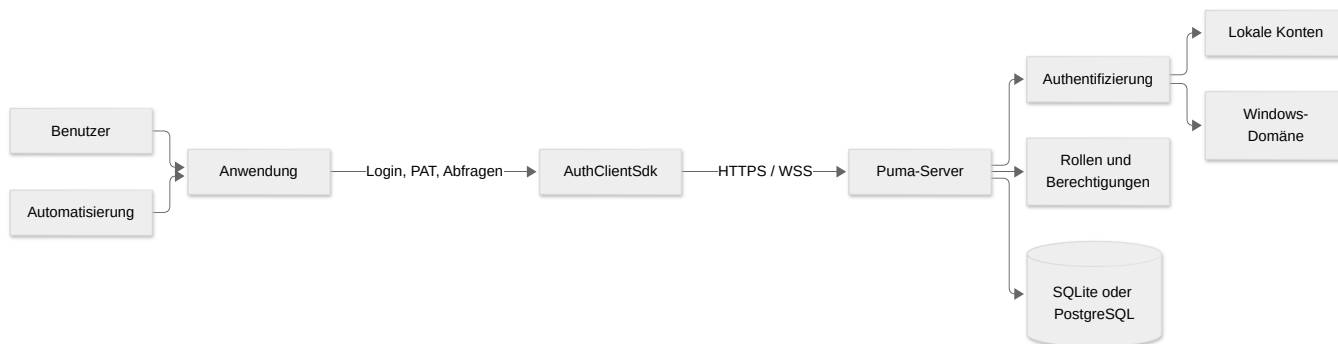


Abbildung 1: Überblick über die Puma-Systemarchitektur.

Puma trennt vier Aufgaben:

1. **Identität:** Wer greift zu?
2. **Authentifizierung:** Ist der vorgelegte Nachweis gültig?
3. **Autorisierung:** Welche produktbezogenen Aktionen sind erlaubt?

4. **Persistenz:** Wo werden Benutzer, Rollen, Gruppen, Sitzungen und PATs gespeichert?

2.1 Varianten des Puma-Servers

Variante	Datenbank	Typischer Einsatz
PumaServerSl	SQLite	Einzelinstallation, Entwicklung, kleine lokale Installation
PumaServerPg	PostgreSQL	Zentraler Mehrbenutzerbetrieb und produktive Serverinstallation

Beide Varianten verwenden dieselbe Puma-Serverbasis. Die jeweilige Serveranwendung bindet die passenden Repositories und SQL-Skripte für ihre Datenbank ein.

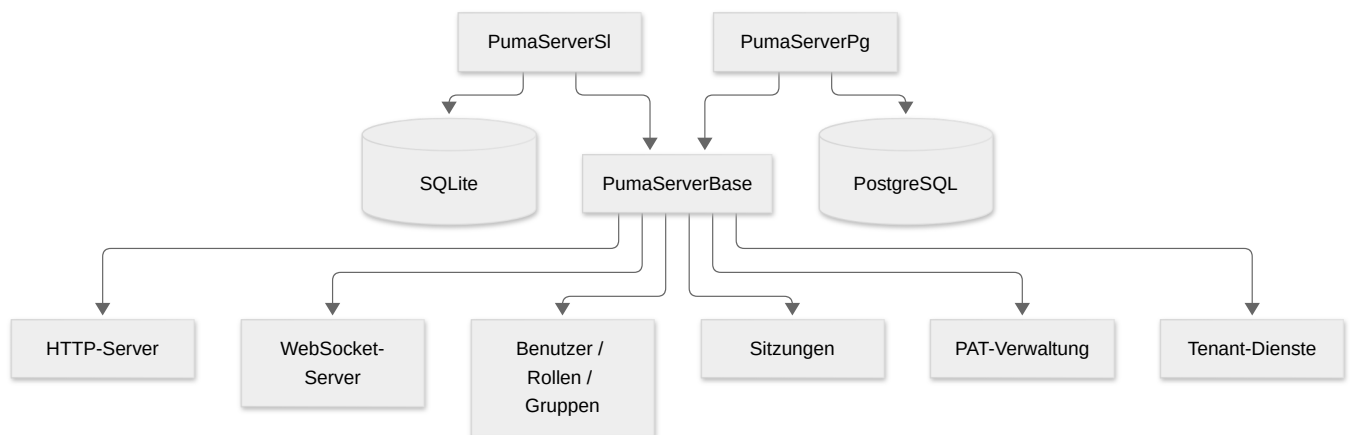


Abbildung 2: Gemeinsame Serverbasis und datenbankspezifische Varianten.

2.2 Zusammenspiel von Puma und Anwendung

Die Anwendung bindet die Puma-Administration als Seite in ihre Client-UI ein. Auf dieser Administrationsseite richten berechtigte Administratoren Benutzer, Rollen, Gruppen und Berechtigungszuordnungen ein. Die Seite greift über das SDK auf den Puma-Server zu; die Daten werden nicht separat in der Anwendung verwaltet. Nach der Anmeldung liefert Puma die effektiven Berechtigungen an die Anwendung zurück. Die Anwendung verwendet sie, um Funktionen in ihrer fachlichen UI freizugeben.

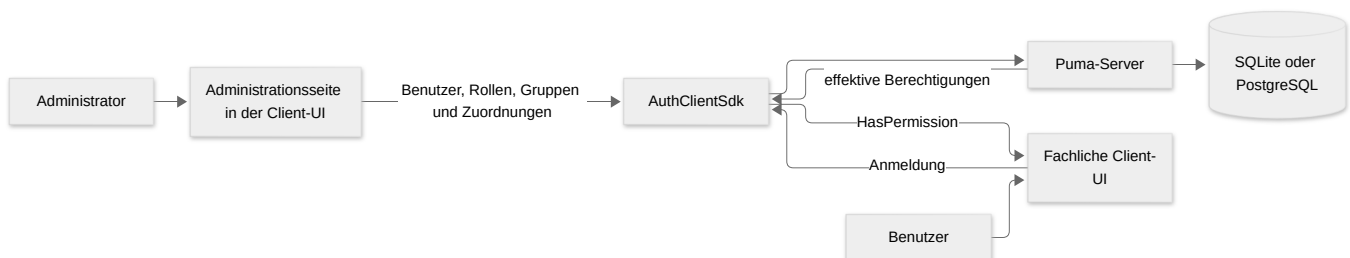


Abbildung 3: Zentrale Administration und Nutzung der Berechtigungen in der Anwendung.

Das Einbinden und berechtigungsabhängige Anzeigen der Administrationsseite ist Aufgabe der Anwendung. Puma erzwingt weiterhin die Berechtigungen für Verwaltungsoperationen; das Ausblenden der Seite allein ist keine Zugriffskontrolle.

3. Rollen- und Berechtigungsmodell

Puma verwaltet Berechtigungen nicht als frei editierbare Eigenschaften eines Benutzers, sondern über Rollen:



Abbildung 4: Beziehungen zwischen Benutzern, Gruppen, Rollen und Berechtigungen.

- **Benutzer** besitzen eine interne, stabile Objekt-ID und einen Login-Namen.
- **Rollen** bündeln Berechtigungen und sind produktspezifisch.
- **Gruppen** bündeln Benutzer und erhalten Rollen.
- **Berechtigungen** sind von der Anwendung definierte, groß- und kleinschreibungsabhängige IDs.
- **Produkt-ID** begrenzt den Kontext, in dem Rollen und Berechtigungen gelten.

Ein Benutzer erhält die Vereinigungsmenge aus:

- Berechtigungen direkt zugeordneter Rollen und
- Berechtigungen der Rollen seiner Gruppen.

Bei mehreren direkten Rollen, mehreren Gruppen oder mehreren Rollen pro Gruppe werden alle enthaltenen Berechtigungen vereinigt. Doppelte Berechtigungen wirken dabei nur einmal; eine Zuordnung zieht keine Berechtigung aus einer anderen Zuordnung ab.

Wichtig: Verwaltungsoperationen verwenden die interne Benutzer-ID, nicht den Login-Namen. Eine Anwendung setzt ihre Produkt-ID vor der Anmeldung.

Die Administrationsseite liest und schreibt Rollen und Berechtigungen ausschließlich im Kontext der aktuellen Produkt-ID. Ist ein Benutzer in mehreren Produkten berechtigt, bleiben die Zuordnungen der übrigen Produkte beim Speichern unverändert.

3.1 Verbindliche und optionale Elemente

Element	Verbindlich oder optional?
Produkt-ID	Verbindlich: Die Anwendung setzt sie vor der Anmeldung; Rollen und Berechtigungen gelten in diesem Produktkontext.
Berechtigungs-ID	Verbindlich für geschützte Funktionen: Die Anwendung definiert und prüft sie.
Rolle	Verbindlich für die Vergabe einer Berechtigung: Berechtigungen werden über Rollen und nicht direkt an Benutzer vergeben. Ein Benutzer darf ohne Rolle bestehen, hat dann aber keine rollenbasierte Berechtigung.
Direkte Rollenzuordnung	Optional: Geeignet für individuelle Aufgaben.
Gruppe	Optional: Geeignet, um wiederkehrende Teamzuordnungen zu bündeln.
Gruppenrolle	Optional: Alternative oder Ergänzung zur direkten Rollenzuordnung.
Mehrere Rollen oder Gruppen	Optional: Ihre Berechtigungen bilden gemeinsam die Vereinigungsmenge.

3.2 Empfohlenes Administrationsmodell

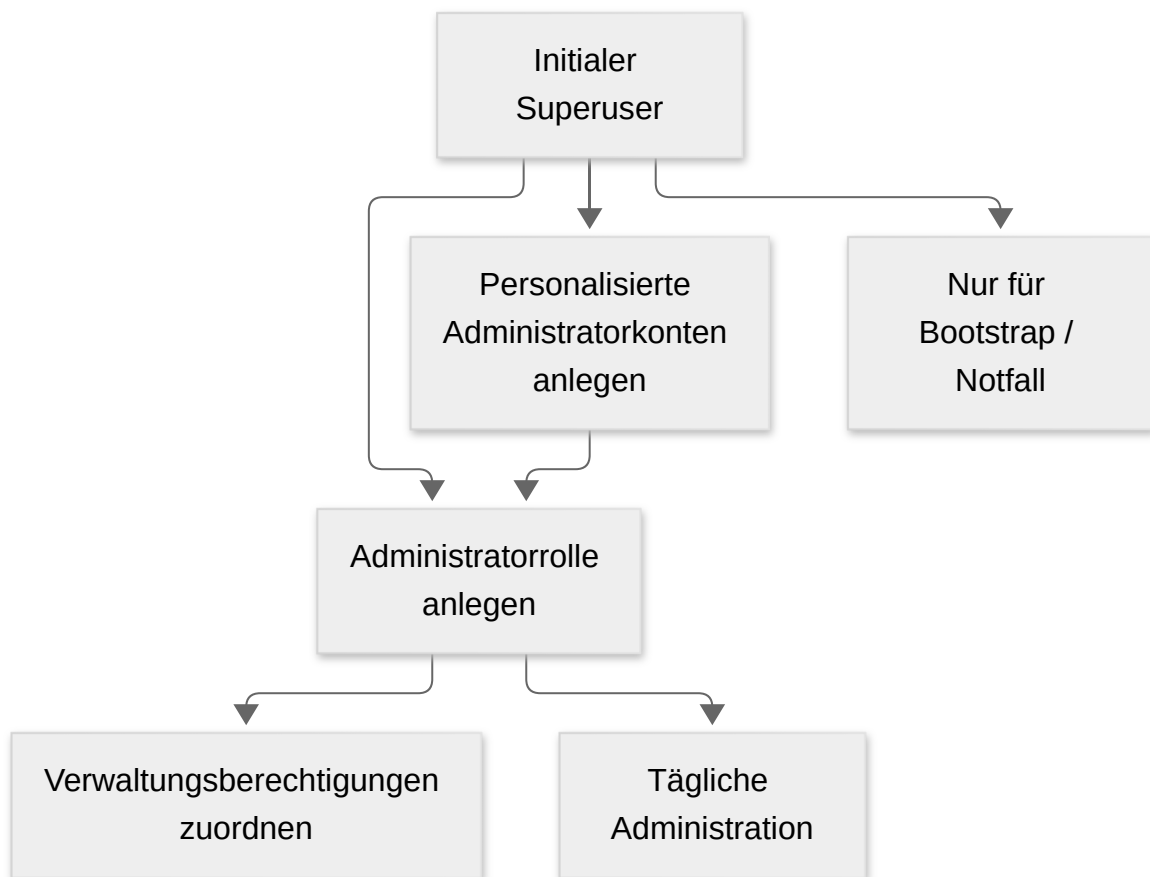


Abbildung 5: Empfohlenes Modell für Bootstrap und tägliche Administration.

Der Superuser dient zum initialen Aufbau. Für den Alltag sollten personalisierte Administratorkonten mit einer passenden Administratorrolle verwendet werden. So müssen

Superuser-Zugangsdaten nicht geteilt werden.

4. Inbetriebnahme des Puma-Servers

4.1 Vorbereitung

1. Servervariante auswählen.
2. Bei `PumaServerPg` PostgreSQL-Datenbank, Datenbankbenutzer und Erreichbarkeit vorbereiten.
3. Bei `PumaServerSl` ist kein separater Datenbankserver und keine Datenbankvorbereitung erforderlich. Der Server legt die SQLite-Datenbank an; das Dienstkonto benötigt Schreibrechte am vorgesehenen Speicherort.
4. HTTP- und WebSocket-Port festlegen.
5. Für produktive Systeme ein Serverzertifikat und einen privaten Schlüssel bereitstellen.
6. Firewall nur für die benötigten Ports öffnen.
7. Schreibrechte für Einstellungen, Datenbank und Protokolle prüfen.

Die persistenten Puma-Einstellungen werden standardmäßig unter dem anwendungsbezogenen Systempfad in `Puma/Puma Server/PumaServerSettings.xml` abgelegt.

4.2 Startablauf

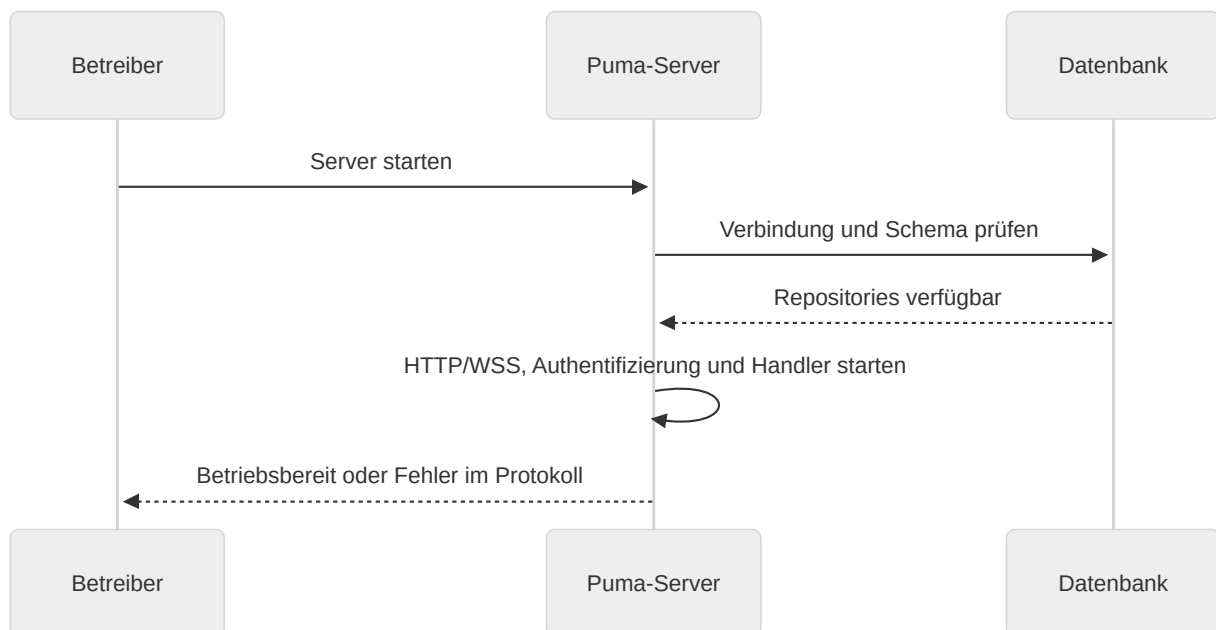


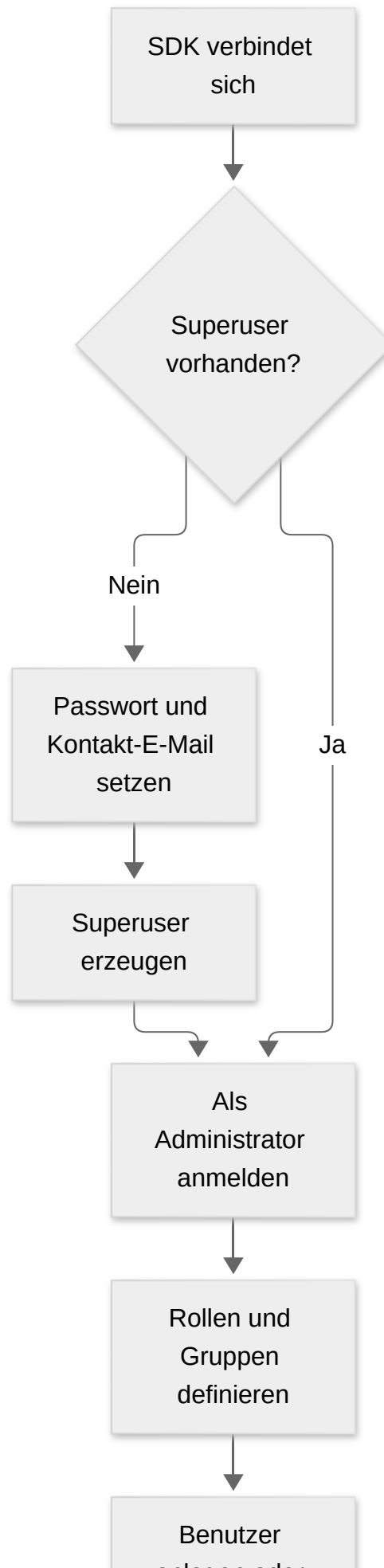
Abbildung 6: Technischer Startablauf des Puma-Servers.

Nach dem Start sind insbesondere zu prüfen:

- Datenbankverbindung erfolgreich,
- HTTP- und WebSocket-Port gebunden,
- Zertifikat und Schlüssel geladen,
- keine Migrations- oder Repositoryfehler,

- Client kann den Server erreichen.

4.3 Ersteinrichtung



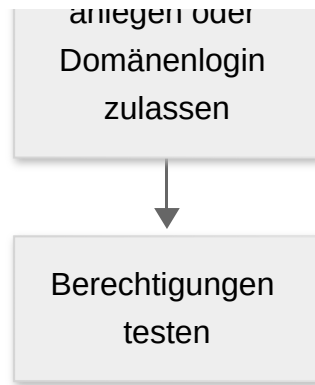


Abbildung 7: Ersteinrichtung vom Superuser bis zum Berechtigungstest.

Der SDK-Ablauf besteht aus `SuperuserExists()`, bei Bedarf `CreateSuperuser()`, danach Login und Aufbau des Rollenmodells. Bei der Initialisierung werden das Passwort des Superusers und seine Kontakt-E-Mail festgelegt. `CreateSuperuser()` übernimmt das Passwort; die Kontakt-E-Mail wird in der Client-Komposition als `SuperuserMail` des `RemoteSuperuserController` konfiguriert und beim Erzeugen übermittelt. Die Puma-Tests verwenden für das initiale Konto den Login `su`; produktive Zugangsdaten und eine erreichbare Kontakt-E-Mail müssen sicher gewählt und verwahrt werden. Das Superuser-Passwort unterliegt bereits der Passwortrichtlinie aus [Abschnitt 9.4](#).

4.4 Transportverschlüsselung

Puma verwendet getrennte Ports für HTTP(S) und WebSocket(S). Sobald Clients außerhalb eines isolierten Entwicklungsrechners zugreifen, sind HTTPS und WSS zu verwenden.

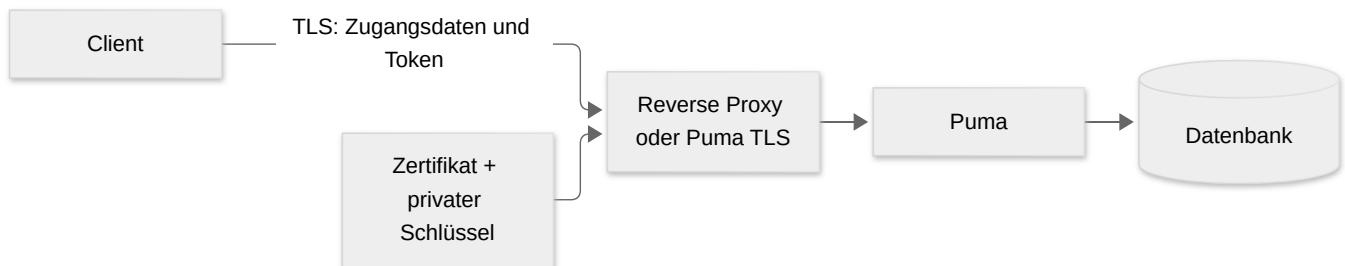


Abbildung 8: TLS-geschützter Transport zwischen Client und Puma.

Produktionsregeln:

- TLS 1.2 oder höher einsetzen.
- Zertifikatsprüfung nicht deaktivieren.
- Privaten Schlüssel nur für den Serverprozess lesbar machen.
- Passphrasen nicht in Quellcode oder Präsentationen speichern.
- HTTP/WS ohne TLS nur in kontrollierten Testumgebungen verwenden.

5. Detaillierte Use Cases

UC-01: Lokalen Benutzer anlegen und berechtigen

Akteur: Administrator

Vorbedingung: Administrator ist angemeldet und besitzt die nötigen Verwaltungsberechtigungen.

1. Benutzer mit Anzeigenname, eindeutigem Login, Startpasswort und E-Mail anlegen.
2. Interne Benutzer-ID aus dem Ergebnis übernehmen.
3. Vorhandene Rolle zuordnen oder zuerst eine Rolle anlegen.
4. Optional den Benutzer einer Gruppe hinzufügen.
5. Benutzer meldet sich an.
6. Anwendung prüft die erwarteten Berechtigungen.

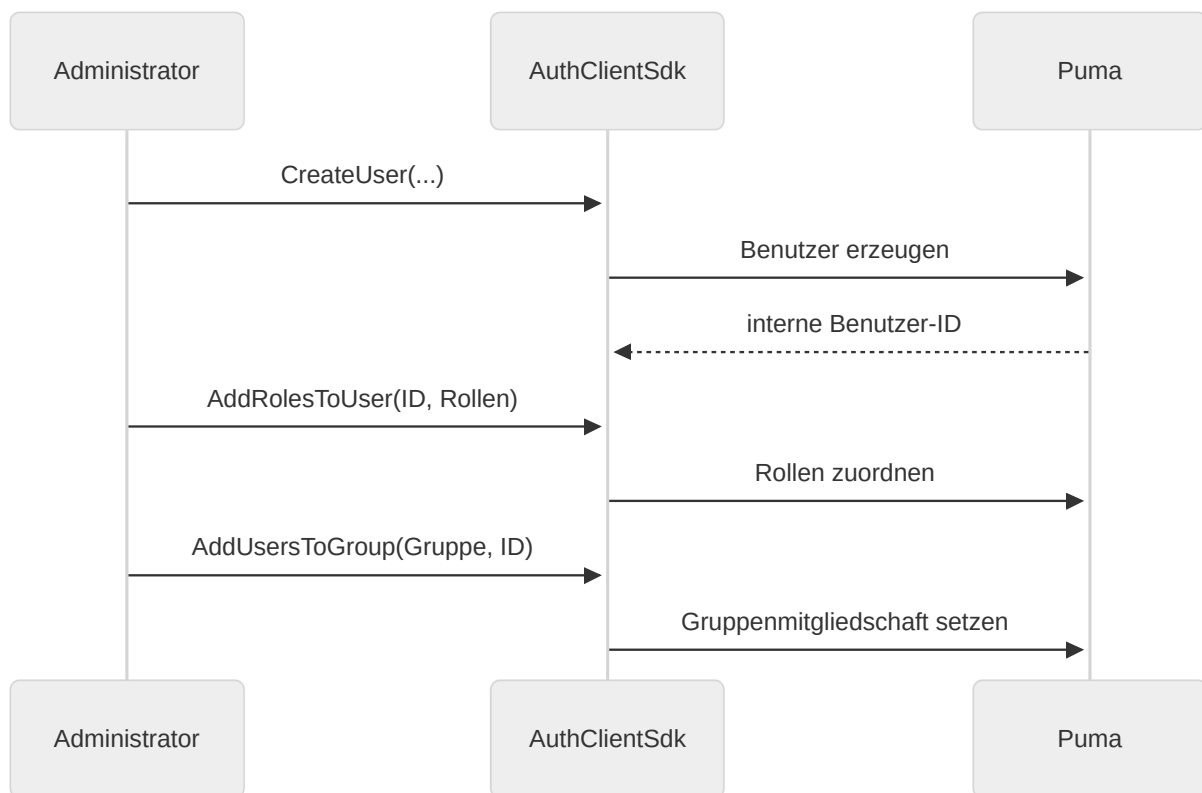


Abbildung 9: Anlage eines lokalen Benutzers mit Rollen- und Gruppenzuordnung.

Ergebnis: Der Benutzer erhält Berechtigungen aus direkten Rollen und Gruppenrollen. Ein bereits angemeldeter Benutzer muss sich gegebenenfalls neu anmelden, damit die Anwendung aktualisierte Sitzungsberechtigungen erhält.

UC-02: Team über eine Gruppe verwalten

Akteur: Administrator

1. Fachliche Rolle mit den benötigten Berechtigungen erstellen.
2. Gruppe für Team oder Abteilung erstellen.

3. Rolle der Gruppe zuordnen.
4. Benutzer zur Gruppe hinzufügen.
5. Beim Austritt Benutzer aus der Gruppe entfernen.

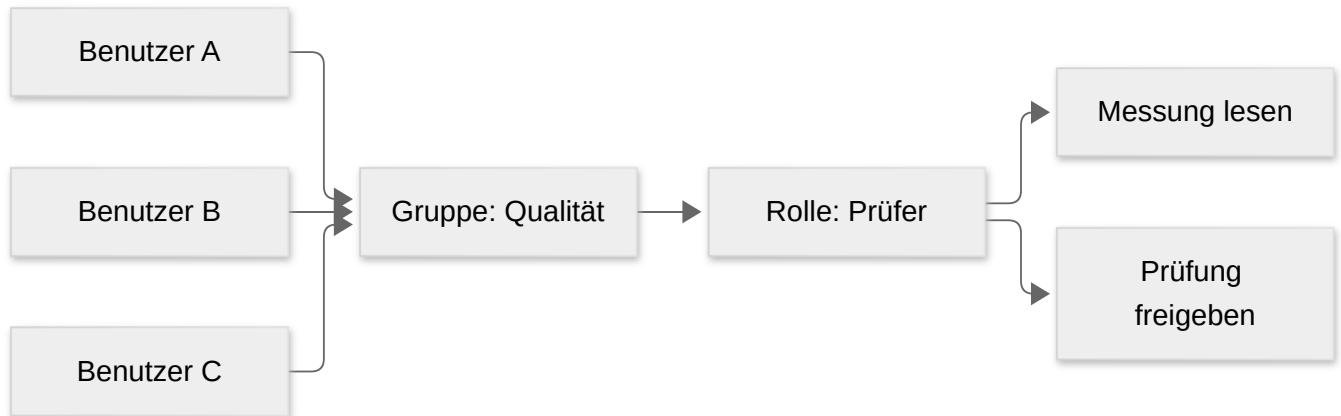


Abbildung 10: Gemeinsame Berechtigungsvergabe über eine Gruppe.

Nutzen: Rollenänderungen wirken zentral auf alle Gruppenmitglieder.

UC-03: Anmeldung und funktionsbezogene Freigabe

Akteur: Endbenutzer

1. Anwendung konfiguriert Verbindung und Produkt-ID.
2. Benutzer gibt Login und Passwort ein.
3. Puma validiert die Zugangsdaten.
4. Puma erzeugt eine Sitzung und liefert Token, Benutzername, Produkt-ID und Berechtigungen.
5. Anwendung gibt nur erlaubte Funktionen frei.
6. Beim Abmelden invalidiert Puma die Sitzung.

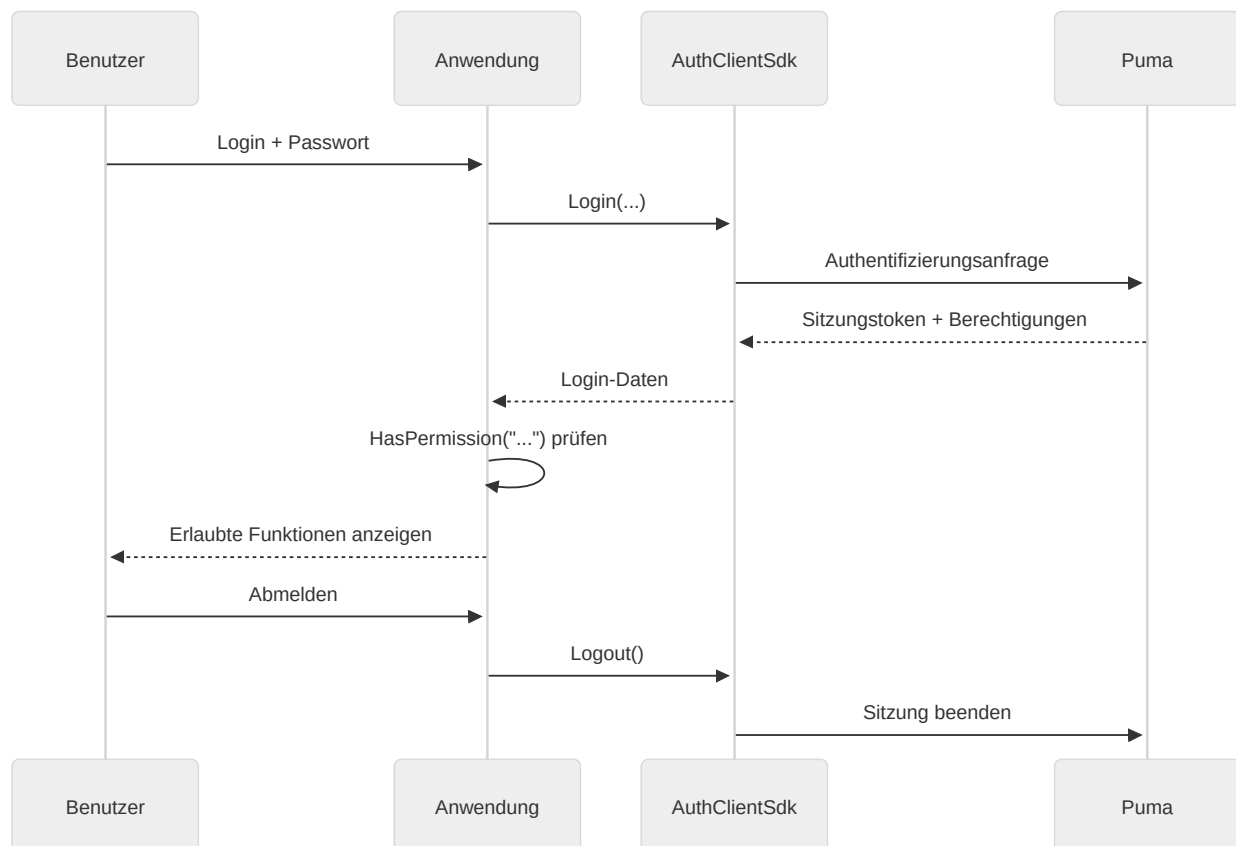


Abbildung 11: Anmeldung, Berechtigungsprüfung und Abmeldung einer Anwendung.

Fehler bei ungültigen Zugangsdaten, fehlender Verbindung oder fehlenden Serverkomponenten werden als fehlgeschlagener Login gemeldet. Ein deaktiviertes Konto wird erst nach erfolgreicher Prüfung der Zugangsdaten mit einer eigenen Meldung abgewiesen, damit der Kontostatus nicht über Anmeldeversuche ausgespielt werden kann (siehe [Abschnitt 9.5](#)).

UC-04: Berechtigungsänderung

1. Administrator ändert Rollen- oder Gruppenzuordnung.
2. Anwendung beendet alte Sitzung oder fordert erneute Anmeldung.
3. Benutzer meldet sich erneut an.
4. Anwendung baut ihre Oberfläche aus den neuen Berechtigungen auf.

Das Ausblenden einer Schaltfläche ersetzt keine serverseitige Prüfung. Jede schutzbedürftige Serveroperation muss die Berechtigung nochmals validieren.

UC-05: Benutzer deaktivieren oder entfernen

Für den dauerhaften Entzug des Zugangs stehen zwei Wege zur Verfügung:

1. **Konto deaktivieren (bevorzugt):** Der Superuser setzt den Kontostatus im Benutzereditor auf deaktiviert. Der Benutzerdatensatz, seine Rollen und Gruppenzuordnungen bleiben erhalten, eine Anmeldung ist aber nicht mehr möglich. Laufende Sitzungen und PATs des Kontos werden sofort abgewiesen. Einzelheiten beschreibt [Abschnitt 9.5](#).

2. **Konto entfernen:** Das Client-SDK stellt `RemoveUser()` als permanente Löschung bereit. Rollen- und Gruppenzuordnungen werden mit dem Benutzer entfernt. Vor dem Löschen sind fachliche Aufbewahrungs- und Audit-Anforderungen zu prüfen.

Soll nur eine einzelne Funktion entzogen werden, genügt es, die entsprechende Rollen- oder Gruppenzuordnung zu entfernen.

UC-06: Eigene Anwendung anbinden

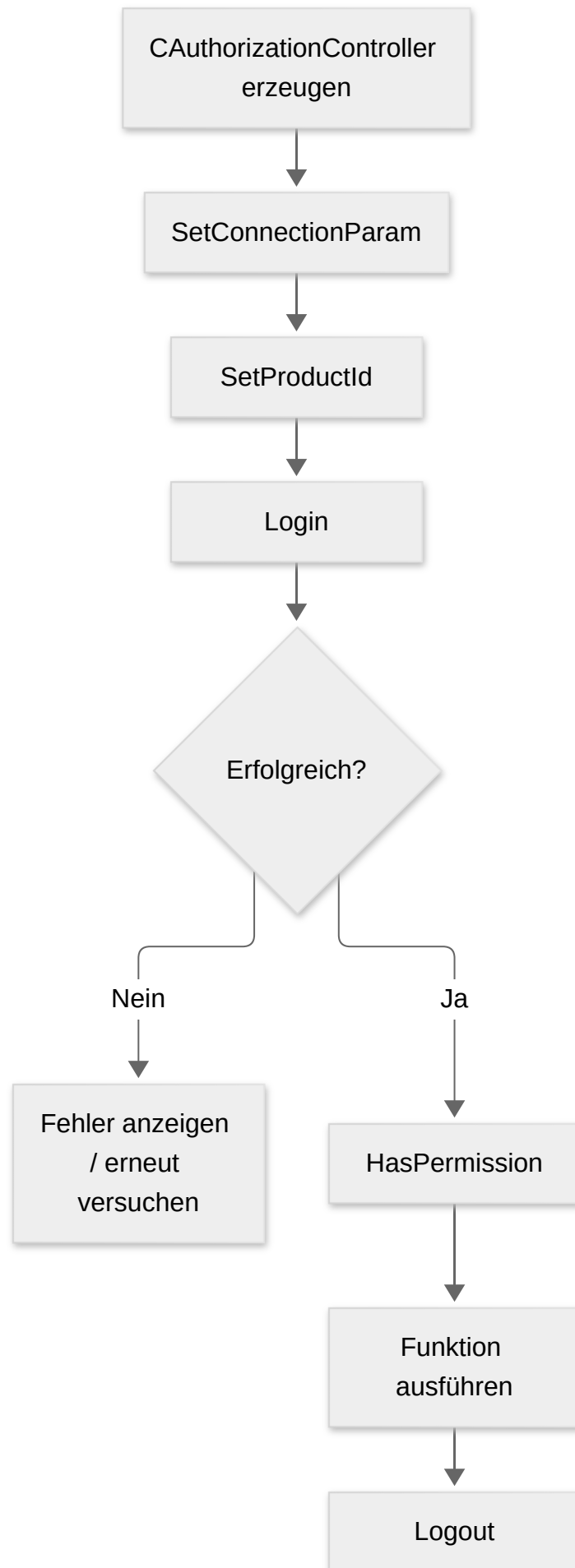


Abbildung 12: Minimaler Ablauf einer SDK-basierten Client-Anbindung.

Die minimale Reihenfolge im C++-Client lautet:

```
AuthClientSdk::CAuthorizationController auth;

AuthClientSdk::ServerConfig server;
server.host = "puma.example.org";
server.httpPort = 443;
server.wsPort = 8443;
server.sslConfig = AuthClientSdk::SslConfig{};

auth.SetConnectionParam(server);
auth.SetProductId("MeineAnwendung");

AuthClientSdk::Login session;
if (auth.Login(login, password, session) &&
    auth.HasPermission("messung.lesen")) {
    // Geschützte Funktion freigeben.
}
auth.Logout();
```

Sicherheitsrelevante Hinweise:

- Passwörter nur über TLS übertragen.
- Sitzungstoken nicht protokollieren.
- `Login()` beendet eine vorherige Sitzung des Controllers automatisch.
- `Logout()` explizit aufrufen; der Destruktor versucht zusätzlich eine Best-Effort-Abmeldung.
- `Login()` und `Logout()` nicht parallel auf demselben Controller ausführen.

UC-07: Eigenen autorisierbaren Server anbinden

`AuthServerSdk::CAuthorizableServer` ist für Serveranwendungen vorgesehen, die eigene Endpunkte anbieten, aber Puma als zentrale Autoritätsquelle verwenden.

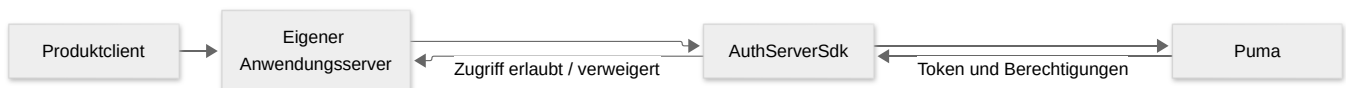


Abbildung 13: Puma-Anbindung eines eigenen autorisierbaren Servers.

Der Anwendungsserver setzt:

1. seine Produkt-ID,
2. die Verbindung zum zentralen Puma-Server,
3. seine eigenen HTTP-/WebSocket-Ports,
4. optional eine Featuredatei und TLS-Konfiguration,
5. anschließend `Start()` und beim Herunterfahren `Stop()`.

6. SDK-Schicht

6.1 AuthClientSdk

Die Fassade `AuthClientSdk::CAuthorizationController` bietet:

Bereich	Zentrale Operationen
Verbindung	<code>SetConnectionParam()</code> , <code>SetProductId()</code>
Sitzung	<code>Login()</code> , <code>Logout()</code> , <code>GetToken()</code>
Autorisierung	<code>HasPermission()</code> , <code>GetTokenPermissions()</code>
Bootstrap	<code>SuperuserExists()</code> , <code>CreateSuperuser()</code>
Benutzer	Auflisten, lesen, anlegen, löschen, Passwort ändern
Rollen	Auflisten, lesen, anlegen, löschen, Berechtigungen zuordnen
Gruppen	Auflisten, lesen, anlegen, löschen, Benutzer/Rollen zuordnen
PAT	Erstellen, auflisten, validieren und widerrufen

`ServerConfig` enthält Host, HTTP-Port, WebSocket-Port und optionale TLS-Einstellungen. Rollen und Berechtigungen sind an die mit `SetProductId()` konfigurierte Anwendung gebunden.

6.2 AuthServerSdk

Das Server-SDK kapselt einen autorisierbaren HTTP-/WebSocket-Server. Seine Netzwerkverbindung zum Puma-Backend ist von den Ports getrennt, auf denen der eigene Server Clients bedient. Bei verteilten Installationen müssen daher beide Verbindungsrichtungen konfiguriert und abgesichert werden.

6.3 UI-Komponenten

Puma enthält Widgets beziehungsweise QML-Komponenten für Login und Administration. Sie bauen auf denselben Authentifizierungs- und Verwaltungsschnittstellen auf. Eine eigene Oberfläche darf die serverseitigen Berechtigungsprüfungen nicht ersetzen.

7. LDAP-/Windows-Domänenanmeldung

7.1 Funktionsweise

Die aktuelle `ImtCore`-Implementierung verwendet auf Windows die Windows-Domänenfunktionen, insbesondere die Prüfung über `LogonUser`. Sie ist damit auf Windows-/Active-Directory-Umgebungen ausgerichtet und kein allgemein konfigurierbarer OpenLDAP-Client.

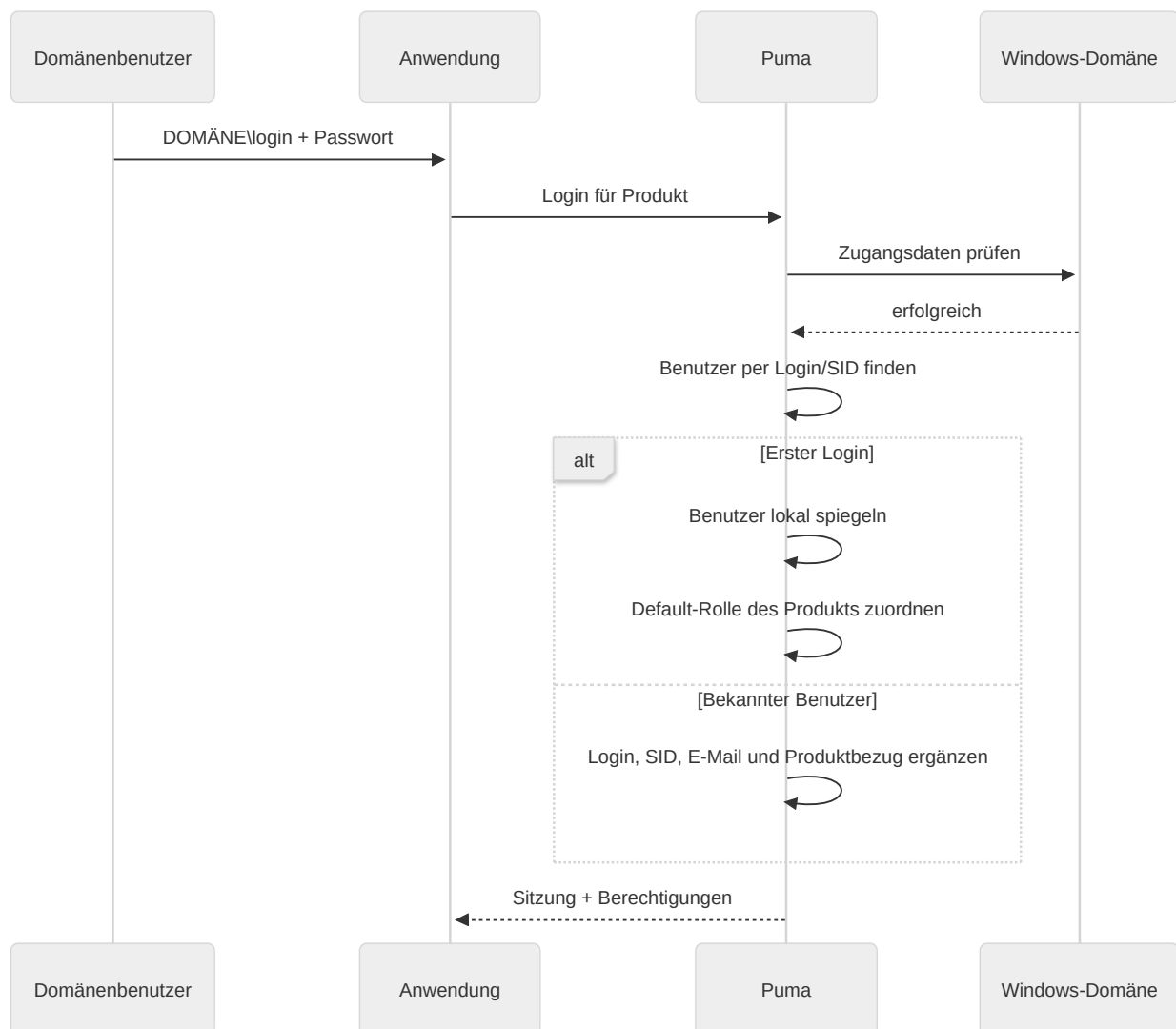


Abbildung 14: Anmeldung eines Windows-Domänenbenutzers über Puma.

Bei erfolgreichem ersten Domänenlogin:

- legt Puma einen internen Benutzerdatensatz an,
- kennzeichnet das Authentifizierungssystem als **LDAP**,
- übernimmt, soweit verfügbar, SID, Anzeigename und E-Mail,
- erzeugt bei Bedarf die produktbezogenen Rollen **Guest** und **Default**,
- weist dem Benutzer für das Produkt die Default-Rolle zu.

Anschließend können Administratoren dem gespiegelt angelegten Benutzer weitere Puma-Rollen und Gruppen zuordnen. Das Passwort wird weiterhin gegenüber der Windows-Domäne geprüft.

7.2 Aktivierung und Deaktivierung

LdapEnabled ist in der Puma-StandardEinstellung aktiviert und wird über den Einstellungsbereich **LDAP** bereitgestellt. Wenn ausschließlich lokale Puma-Konten verwendet werden, sollte die Funktion deaktiviert werden, um unnötige Domänenprüfungen und irreführende Meldungen zu vermeiden.

7.3 Voraussetzungen

- Puma läuft unter Windows.
- Der Server kann die Domäne und einen Domain Controller erreichen.
- Betriebssystem, DNS und Vertrauensstellung sind korrekt konfiguriert.
- Benutzer verwendet einen von Windows akzeptierten Login, typischerweise `DOMÄNE\benutzer`.
- LDAP ist in Puma aktiviert.

7.4 Fehlersuche

Symptom	Prüfung
Domänenlogin scheitert, lokaler Login funktioniert	Domänen erreichbarkeit, DNS, Uhrzeit, Loginformat und <code>LdapEnabled</code> prüfen
Benutzer wird doppelt angelegt	Einheitliches Loginformat und SID-Auflösung prüfen
Benutzer hat nach erstem Login zu wenig Rechte	Default-Rolle und weitere Rollen-/Gruppenzuordnung prüfen
Lokale Logins erzeugen Domänenfehler im Protokoll	LDAP deaktivieren, wenn nicht benötigt
Linux-Server authentifiziert nicht gegen AD	Aktuelle Implementierung ist Windows-spezifisch

8. Personal Access Tokens (PAT)

8.1 Einsatz

PATs sind langlebige Zugangsnachweise für Automatisierung, CI/CD, Überwachungsdienste und Service-zu-Service-Kommunikation. Ein PAT gehört zu einem Benutzer, enthält eine Produkt-ID und explizite Berechtigungssscopes.

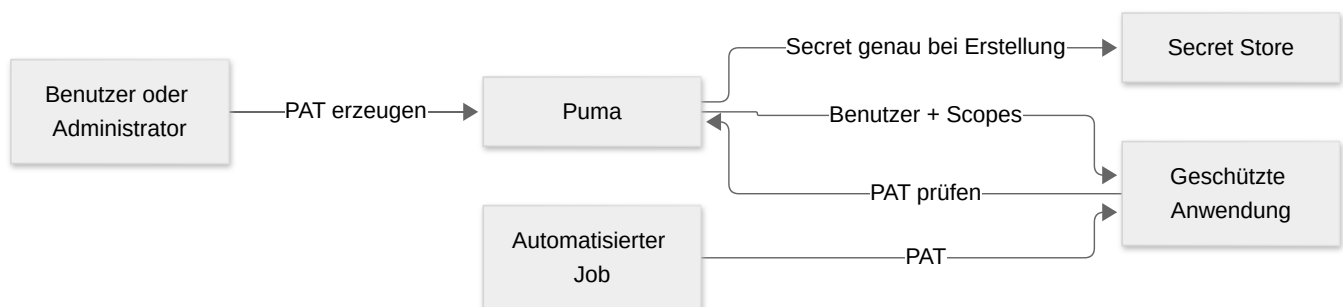


Abbildung 15: Erstellung, Speicherung und Verwendung eines PAT.

8.2 Lebenszyklus

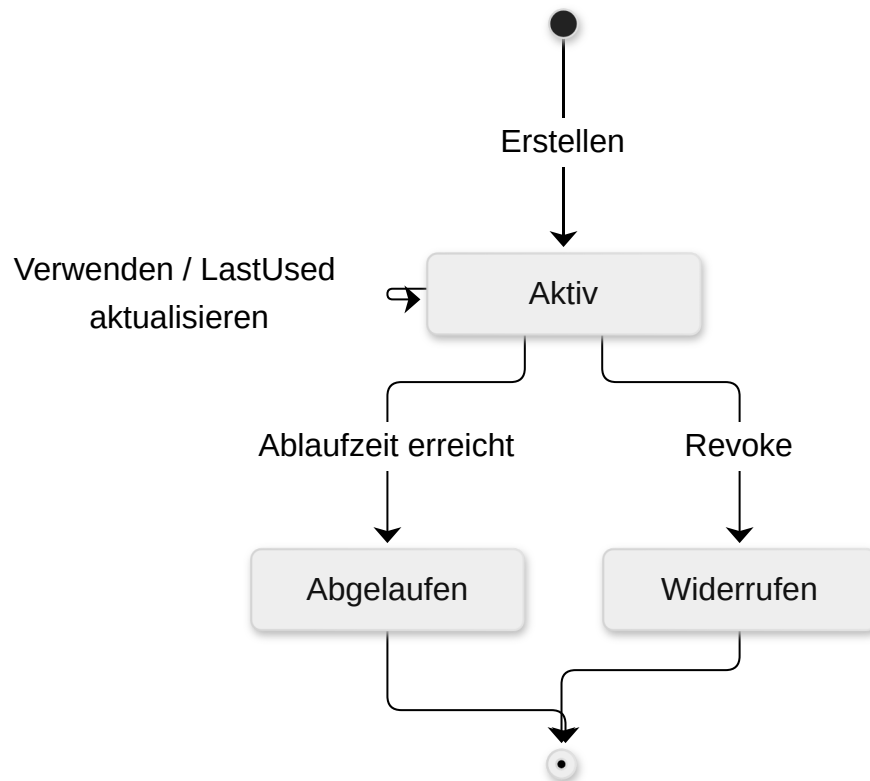


Abbildung 16: Zustände im Lebenszyklus eines PAT.

Ein Token ist gültig, wenn es existiert, aktiv, nicht widerrufen und nicht abgelaufen ist. Widerrufene Datensätze bleiben in der Liste sichtbar und werden als inaktiv gemeldet.

8.3 PAT erstellen

Vorbedingung: Der Eigentümer oder ein Administrator ist mit einer Sitzung angemeldet.

1. Zweckbezogenen Namen vergeben, zum Beispiel `CI Produktion Lesen`.
2. Zielbenutzer und Produkt-ID festlegen.
3. Nur die minimal nötigen Scopes auswählen.
4. Möglichst ein Ablaufdatum im ISO-8601-Format setzen.
5. Secret unmittelbar in einem Secret Store speichern. Es beginnt mit dem in der Serverkomposition konfigurierten Präfix, standardmäßig `imt_pat_`.
6. Secret nicht in Quellcode, Build-Protokolle oder Tickets kopieren.

Anonyme Aufrufer dürfen keine PATs erstellen. Ein normaler Benutzer kann eigene PATs verwalten, aber nicht die PATs anderer Benutzer. Administratoren können PATs anderer Benutzer verwalten.

8.4 PAT verwenden

Das SDK-Datenmodell unterscheidet `TokenType::Session` und `TokenType::PersonalAccessToken`. Für einen nicht interaktiven Zugriff wird das PAT über `ValidatePersonalAccessToken()` geprüft; die Anwendung verwendet anschließend ausschließlich die zurückgegebenen Scopes und prüft zusätzlich den Produktkontext.

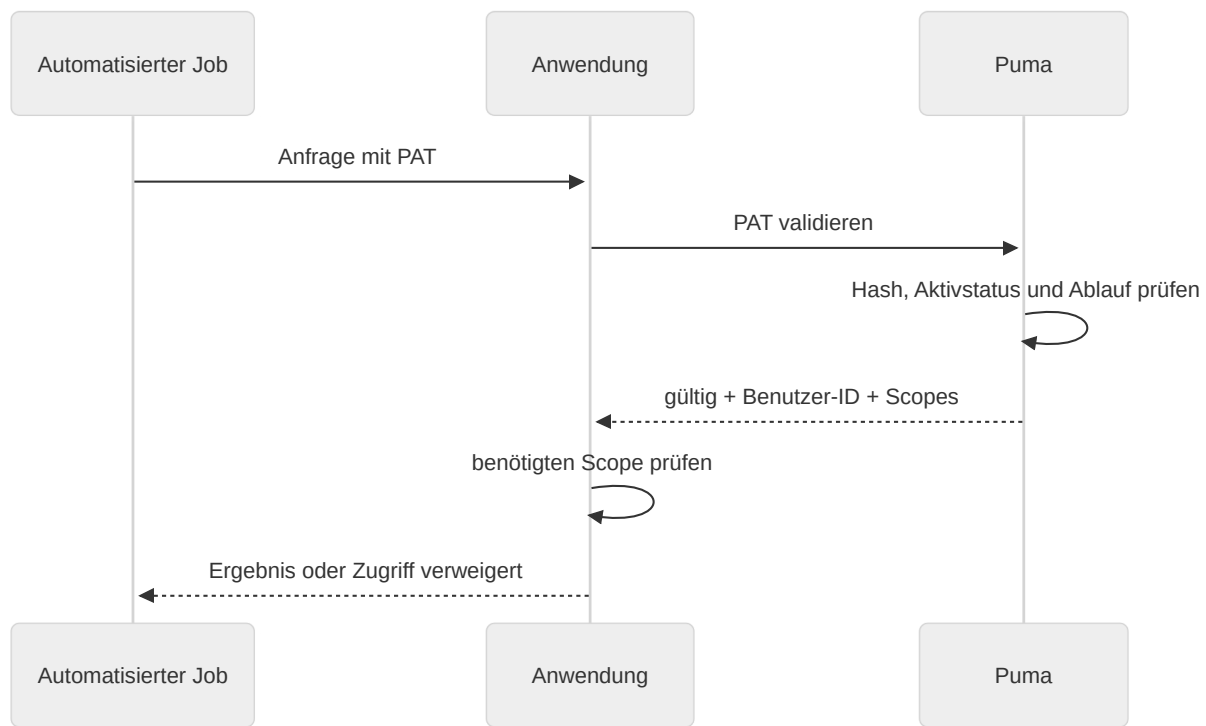


Abbildung 17: Validierung eines PAT für einen automatisierten Zugriff.

`GetTokenPermissions()` akzeptiert inzwischen sowohl Sitzungstoken als auch PATs. Für ein PAT liefert der Server die Schnittmenge aus den Berechtigungen des Benutzers und den Scopes des Tokens und berücksichtigt dabei die Produktbindung des Tokens. Berechtigungen, die der Benutzer zwar über eine Rolle besitzt, die aber nicht im Scope des Tokens liegen, werden nicht zurückgegeben. Wird ein Konto deaktiviert, werden auch seine PATs abgewiesen.

8.5 PAT widerrufen

1. Token anhand von Name, Produkt, Erstellungszeit und letzter Nutzung identifizieren.
2. Token-ID widerrufen.
3. Validierung muss danach fehlschlagen.
4. Bei vermutetem Secret-Abfluss abhängige Systeme und Protokolle prüfen.
5. Ersatz-PAT mit reduziertem Scope und neuem Ablaufdatum ausstellen.

8.6 Bekannte Schnittstelleneigenschaft

Die aktuelle GraphQL-Validierungsantwort liefert Benutzer-ID und Scopes, aber nicht die Token-ID. Deshalb kann `ValidatePersonalAccessToken()` die `productId` im Validierungsergebnis derzeit nicht rekonstruieren. Der Produktkontext muss beim ausstellenden beziehungsweise konsumierenden System zusätzlich bekannt und geprüft sein.

9. Betrieb und Sicherheit

9.1 Verantwortlichkeiten

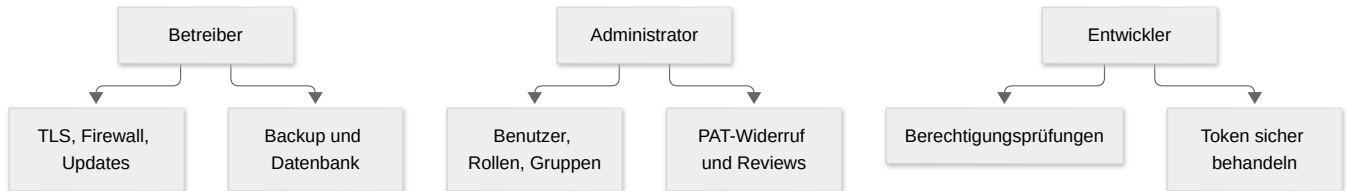


Abbildung 18: Aufteilung der betrieblichen Sicherheitsverantwortung.

9.2 Regelmäßige Kontrollen

- Benutzer ohne aktuellen fachlichen Bedarf deaktivieren oder entfernen.
- Rollen und Gruppen nach dem Least-Privilege-Prinzip prüfen.
- Alte, nie verwendete oder abgelaufene PATs widerrufen.
- Administratorrechte personenbezogen vergeben.
- Datenbank und Einstellungen sichern; Wiederherstellung testen.
- Zertifikatsablauf überwachen.
- Fehlgeschlagene Logins und ungewöhnliche Tokenverwendung untersuchen.
- Server und ImtCore/Puma-Komponenten aktuell halten.

9.3 Backup und Wiederherstellung

Ein konsistentes Backup umfasst mindestens Datenbank und Puma-Einstellungen. Zertifikate und Schlüssel sind getrennt und besonders geschützt zu sichern. Nach einer Wiederherstellung sind Datenbankmigrationen, Login, Rollen, Gruppen, Sitzungsbehandlung und PAT-Validierung in einer kontrollierten Umgebung zu testen.

9.4 Passwortrichtlinie

Puma prüft Passwörter beim Anlegen eines Benutzers, beim Ändern eines Passworts und bei der Initialisierung des Superusers gegen die Passwortrichtlinie der ImtCore-Benutzerverwaltung. Sie ist in der ausgelieferten Puma-Komposition bereits verdrahtet und wirkt ohne weitere Konfiguration mit diesen Vorgaben:

Regel	Standardwert
Mindestlänge	8 Zeichen
Maximallänge	128 Zeichen
Klein-, Groß-, Ziffern- und Sonderzeichenpflicht	jeweils nicht erzwungen
Login als Passwort	abgelehnt
Blockliste bekannter Passwörter	nicht gesetzt; optional als Datei hinterlegbar
Passworthistorie	letzte 5 Passwörter dürfen nicht wiederverwendet werden
Mindestalter des Passworts	0 Tage, also keine Wartezeit vor der nächsten Änderung
Höchstalter des Passworts	0 Tage, also kein Ablauf
Warnfrist vor Ablauf	14 Tage

Weitere Hinweise:

- Eine Ablehnung nennt die verletzten Regeln, zum Beispiel Mindestlänge, fehlende Zeichenklasse, Login als Passwort, Eintrag in der Blockliste, Wiederverwendung oder Mindestalter. Die Administrationsoberfläche zeigt sie als Klartextmeldung an.
- Ein Leerzeichen zählt nicht als Sonderzeichen.
- Ein Administrator setzt das Passwort eines anderen Benutzers ohne dessen altes Passwort und unterliegt dabei nicht dem Mindestalter.
- Für Benutzer aus der Windows-Domäne gilt die Richtlinie nicht; ihr Passwort wird in der Domäne verwaltet und dort geprüft.
- Die Ablaufprüfung bei der Anmeldung und die Abfrage der Regeln durch den Client sind serverseitig vorhanden, in der ausgelieferten Puma-Komposition jedoch nicht mit dem Anmeldecontroller verbunden. Sie wirken erst, wenn die Richtlinie dort zusätzlich verdrahtet wird.

9.5 Kontostatus aktivieren und deaktivieren

Jedes Konto besitzt zusätzlich zu seinen Rollen einen Kontostatus. Ein deaktiviertes Konto bleibt mit allen Zuordnungen erhalten, kann sich aber nicht anmelden.

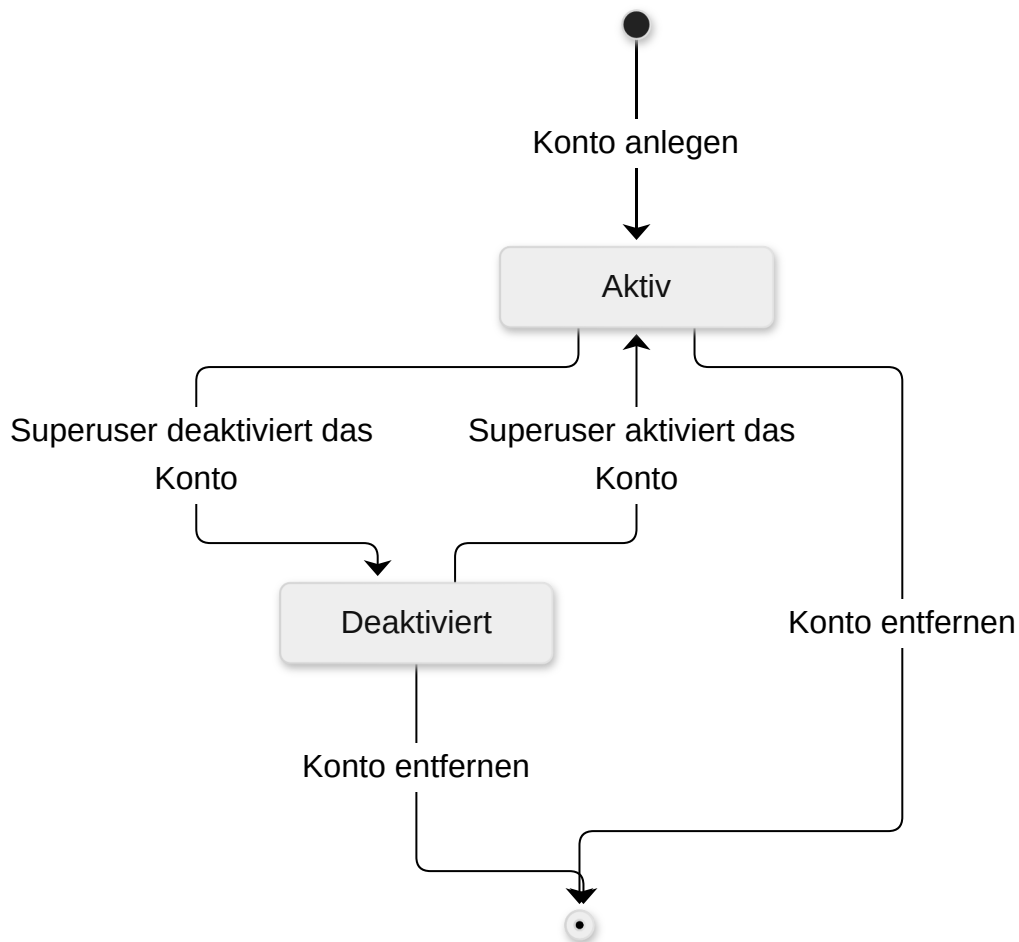


Abbildung 19: Kontostatus und die vom Superuser ausgelösten Übergänge.

Regeln:

- Nur der Superuser sieht und bedient den Schalter für den Kontostatus im Benutzereditor. Für alle anderen Administratoren wird er nicht angezeigt, und ihre Änderungen am Benutzer lassen den Status unverändert.
- Nur der Superuser kann ein Konto bereits deaktiviert anlegen. Eine Selbstregistrierung erzeugt immer ein aktives Konto.
- Bei der Anmeldung werden zuerst die Zugangsdaten geprüft. Erst danach führt ein deaktiviertes Konto zu einer eigenen Meldung, dass das Konto deaktiviert wurde. Ein falsches Passwort liefert weiterhin die allgemeine Meldung, sodass der Kontostatus nicht über Anmeldeversuche ermittelt werden kann.
- Laufende Sitzungen und PATs eines deaktivierten Kontos werden ab sofort abgewiesen; eine Abmeldung ist dafür nicht erforderlich.
- Konten aus älteren Datenbeständen ohne gespeicherten Status gelten als aktiv.

Das Deaktivieren ist dem Löschen vorzuziehen, wenn Zuordnungen, Nachweise oder Zuständigkeiten erhalten bleiben sollen.

9.6 Sperre nach fehlgeschlagenen Anmeldeversuchen

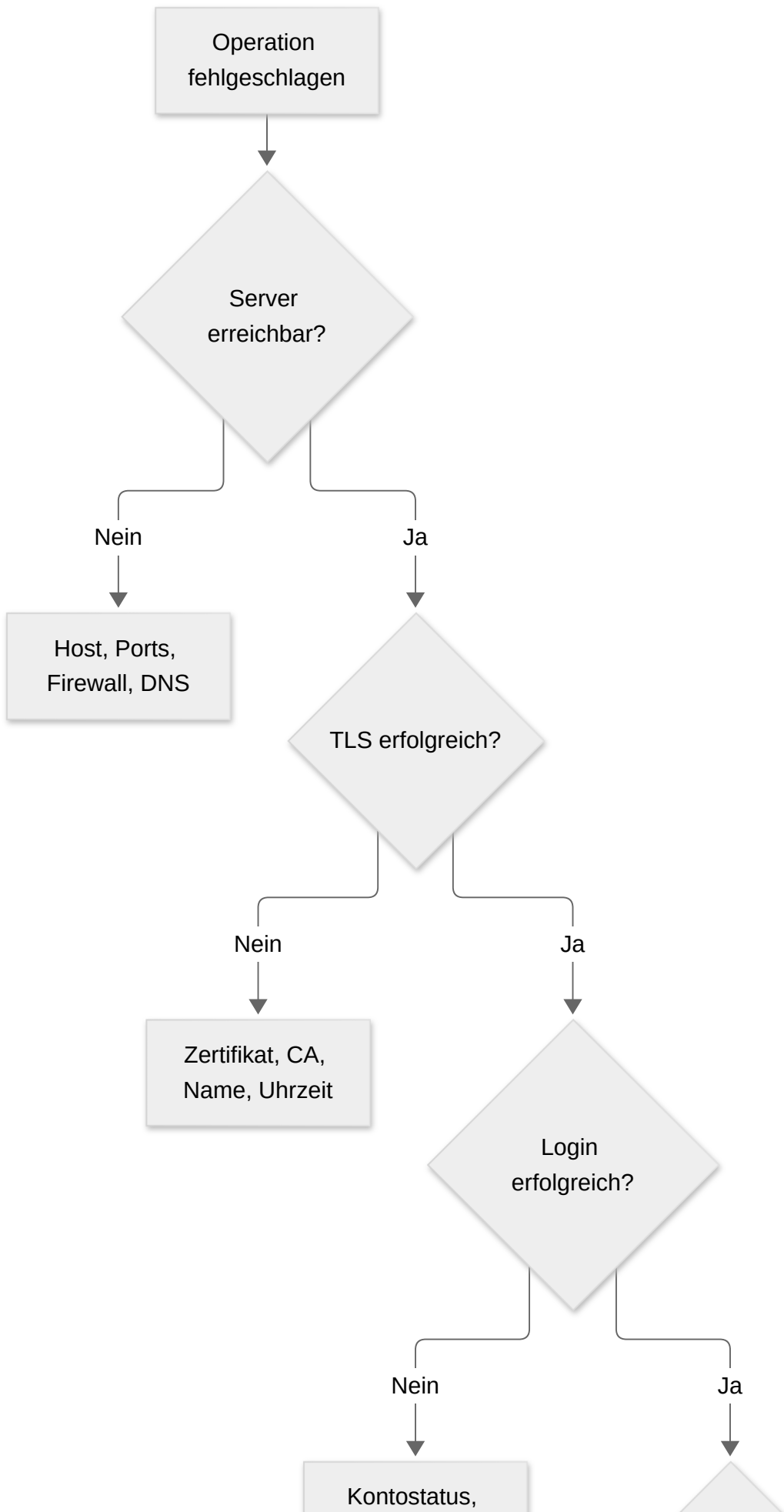
ImtCore stellt eine Kontosperre bereit, die ein Konto nach mehreren aufeinanderfolgenden Fehlversuchen vorübergehend abweist. Ihre Vorgaben lauten:

Einstellung	Standardwert	Bedeutung
Maximale Fehlversuche	5	Anzahl aufeinanderfolgender Fehlversuche bis zur Sperre; 0 schaltet die Sperre ab
Beobachtungszeitraum	300 Sekunden	Zeitfenster, in dem Fehlversuche gezählt werden
Sperrdauer	900 Sekunden	Dauer der Abweisung; 0 bedeutet Sperre bis zur Entsperrung durch einen Administrator

Die Zähler und Sperren werden im Arbeitsspeicher gehalten und beim Neustart des Servers zurückgesetzt. Ein Administrator kann ein Konto vorzeitig entsperren. Der Anmeldeweg über die Windows-Domäne wird nicht mitgezählt.

Hinweis: In der ausgelieferten Puma-Komposition ist die Sperrkomponente nicht verdrahtet; eine automatische Sperre findet daher nicht statt. Wird sie benötigt, ist sie in der Serverkomposition zu ergänzen. Bis dahin sind fehlgeschlagene Anmeldungen über die Protokolle zu überwachen (siehe [Abschnitt 9.2](#)).

10. Fehlerdiagnose



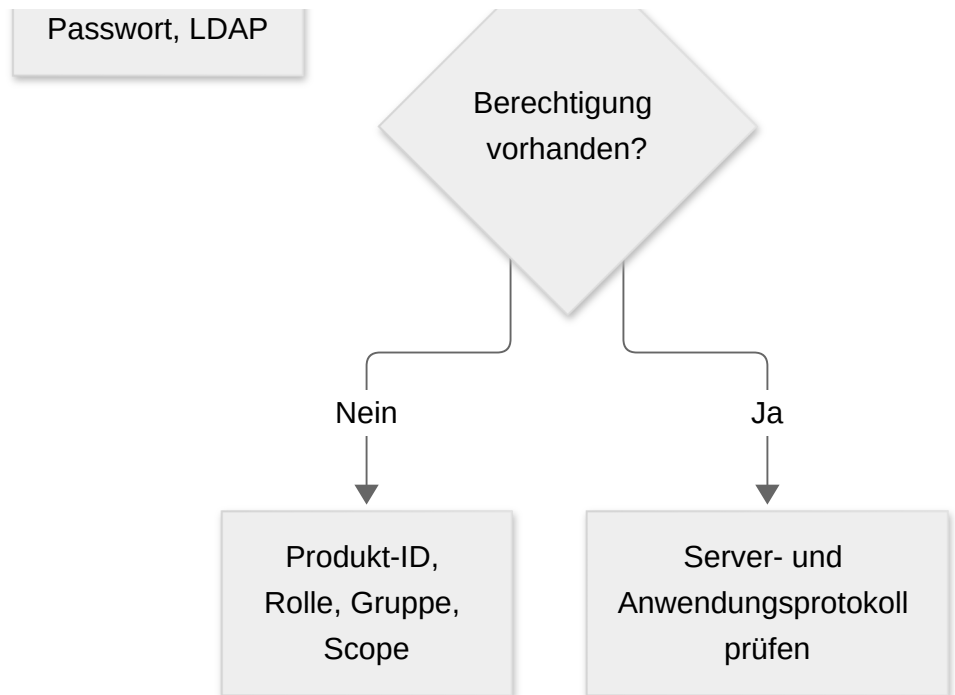


Abbildung 20: Entscheidungsbaum zur Fehlerdiagnose.

Problem	Wahrscheinliche Ursache	Maßnahme
Verbindung abgelehnt	Falscher Host/Port oder Server nicht gestartet	HTTP- und WS-Port sowie Prozess prüfen
TLS-Fehler	Zertifikat nicht vertrauenswürdig oder Name falsch	Zertifikatskette, Hostname und Uhrzeit prüfen
Login schlägt fehl	Zugangsdaten, Kontostatus oder LDAP	Authentifizierungsweg gezielt prüfen
Login meldet ein deaktiviertes Konto	Konto wurde deaktiviert	Kontostatus durch den Superuser prüfen lassen (Abschnitt 9.5)
Passwortänderung wird abgelehnt	Verstoß gegen die Passwortrichtlinie	Gemeldete Regeln auswerten (Abschnitt 9.4)
<code>HasPermission()</code> bleibt <code>false</code>	Falsche Produkt-ID oder fehlende Rolle	Produkt-ID und effektive Rollen prüfen
Benutzeroperation liefert leere ID	Login bereits vorhanden oder Rechte fehlen	Eindeutigkeit und Adminrechte prüfen
PAT-Erstellung liefert leeres Secret	Nicht angemeldet, falscher Eigentümer oder leere Scopes	Sitzung, Benutzer-ID und Scopes prüfen
PAT ist ungültig	Widerrufen, abgelaufen oder verändert	Tokenmetadaten prüfen und neu ausstellen
Einstellungen gehen verloren	Fehlende Schreibrechte	Pfad und Dienstkonto prüfen

11. Abnahme-Checklisten

Server

- ☐ Passende Datenbankvariante gewählt
- ☐ Datenbankverbindung und Migration erfolgreich
- ☐ HTTPS und WSS mit gültigem Zertifikat aktiv
- ☐ Ports und Firewall dokumentiert
- ☐ Backup und Wiederherstellung getestet
- ☐ Protokollüberwachung eingerichtet

Berechtigungsmodell

- ☐ Eindeutige Produkt-ID festgelegt
- ☐ Berechtigungs-IDs dokumentiert
- ☐ Rollen nach Aufgaben statt Personen modelliert
- ☐ Gruppen für wiederkehrende Teams angelegt
- ☐ Personalisierte Administratorkonten eingerichtet
- ☐ Verfahren zum Deaktivieren nicht mehr benötigter Konten festgelegt
- ☐ Passwortrichtlinie geprüft und den Benutzern bekannt gemacht
- ☐ Negativtests für verweigerte Aktionen durchgeführt

LDAP

- ☐ Windows- und Domänenvoraussetzungen erfüllt
- ☐ Erster Domänenlogin getestet
- ☐ SID und Benutzerdaten korrekt übernommen
- ☐ Default-Rolle geprüft
- ☐ LDAP deaktiviert, falls nicht benötigt

PAT

- ☐ Least-Privilege-Scopes vergeben
- ☐ Ablaufdatum gesetzt
- ☐ Secret nur im Secret Store gespeichert
- ☐ Widerruf getestet
- ☐ Rotation und Verantwortlicher dokumentiert

12. Weiterführende Dokumentation

- [AuthClientSdk-Referenz](#)
- [AuthServerSdk-Referenz](#)
- [Abhängigkeiten](#)
- [Puma-Sicherheitsrichtlinie](#)
- [Kompaktpräsentation](#)

13. Entwickleranbindung

Eine Anwendung kann Puma auf zwei Arten einbinden:

Variante	Geeignet für	Abstraktion
AuthClientSdk	Anwendungen, die eine stabile C++-Fassade benötigen	<code>AuthClientSdk::CAuthorizationController</code>
Partitura	ACF-/ImtCore-Anwendungen mit einem autorisierbaren Server	<code>AuthorizableServerFramework.acc</code> aus ImtCore

In beiden Varianten benötigt die Anwendung die Adresse des Puma-Servers, eine eindeutige Produkt-ID und die für das Produkt definierten Berechtigungen. Die Produkt-ID muss mit der administrierten Anwendung übereinstimmen. TLS ist für produktive Verbindungen zu verwenden.

13.1 Einbindung über das SDK

1. `AuthClientSdk` bauen und zur Anwendung linken. Das CMake-Beispiel unter `Impl/AuthClientSdk/CMake/CMakeLists.txt` zeigt die benötigten Qt- und ImtCore-Abhängigkeiten.
2. `AuthClientSdk/AuthClientSdk.h` einbinden.
3. Einen `CAuthorizationController` erzeugen, mit `SetConnectionParam()` den HTTP-/WebSocket-Endpunkt und die TLS-Konfiguration setzen und anschließend mit `SetProductId()` den Produktkontext festlegen.
4. Mit `Login()` anmelden und Funktionen nur nach erfolgreicher Prüfung durch `HasPermission()` freigeben.
5. Für die Administrationsseite die Benutzer-, Rollen- und Gruppenoperationen derselben Fassade verwenden. Die Anwendung entscheidet anhand der Administrationsberechtigung, ob sie die Seite in der Client-UI anbietet.
6. Beim Sitzungsende `Logout()` aufrufen.

Ein minimales C++-Beispiel befindet sich in [UC-06](#); die vollständige API beschreibt die [AuthClientSdk-Referenz](#). Das SDK wird im aktuellen CMake-Build nur unter Windows eingebunden.

13.2 Einbindung über Partitura

ImtCore stellt unter `Partitura/ImtHttpServerVoce.arp/AuthorizableServerFramework.acc` bereits eine fertige Basiskomposition für einen autorisierbaren Server bereit. Sie bündelt unter anderem den HTTP- und WebSocket-Server, die Verbindung zu Puma, den Authentifizierungsmanager sowie die Benutzer-, Rollen- und Gruppen-Caches. Die Anwendung soll diese Basis verwenden, statt die Komponenten einzeln nachzubauen.

Die Integration erfolgt in diesen Schritten:

1. Die ImtCore-Pakete und -Registries in der ACF-Konfiguration der Anwendung bekannt machen.
2. `AuthorizableServerFramework` aus dem Paket `ImtHttpServerVoce` instanziiieren.

3. Die anwendungsspezifischen Komponenten für Anwendungs- und Versionsinformationen, Datenbank, Puma-Verbindung, Server-Schnittstellen und TLS-Konfiguration per `Type="Reference"` anschließen. Eigene Request-Handler werden als `Type="Factory"` angebunden.
4. Produkt-ID und Verbindung zum zentralen Puma-Server konfigurieren. Die Produkt-ID muss mit der in Puma administrierten Anwendung übereinstimmen.
5. Die eigenen GraphQL-Handler mit dem Framework verbinden und die vom Framework exportierten HTTP- und WebSocket-Server in den Server-Controller der Anwendung einbinden.
6. Nach dem Build Anmeldung, PAT- und Sitzungsprüfung sowie erlaubte und verweigerte Berechtigungen gegen eine Testinstanz von Puma prüfen.

`Impl/AuthServerSdk/AuthServerSdk.ac` zeigt eine konkrete Einbindung dieser `ImtCore`-Basiskomposition. Die Partitura-Variante verdrahtet sie deklarativ in der Anwendung, während die SDK-Variante die Integration hinter einer C++-Fassade kapselt.